

Algorytmy z powracaniem

Maria Linkowska 167906, Sara Rachmańczuk 167898

1. Cel zadania

Głównym celem ćwiczenia było zaimplementowanie i zbadanie efektywności algorytmów rozwiązujących dwa klasyczne problemy teorii grafów: poszukiwanie cyklu Eulera oraz poszukiwanie cyklu Hamiltona. Badania obejmowały zarówno grafy nieskierowane, jak i skierowane. Wymagane było zaimplementowanie wariantów decyzyjnych (weryfikujących twierdzenia) oraz wariantów poszukiwań wykorzystujących metodę z powracaniem (backtracking). Dodatkowym aspektem badawczym była ocena wpływu reprezentacji maszynowej grafu na wydajność poszczególnych algorytmów.

2. Opis algorytmów - zastosowano dwie odrębne reprezentacje maszynowe: macierz grafu i macierz sąsiedztwa, i zaimplementowane następujące algorytmy:

- 2.1. **DEC (Decyzyjny – Euler):** Weryfikuje warunek konieczny i wystarczający. Dla grafu nieskierowanego sprawdza parzystość stopni i spójność, dla skierowanego sprawdza równość stopni wchodzących/wychodzących i silną spójność.
- 2.2. **DHC (Decyzyjny – Hamilton):** Sprawdza warunki wystarczające. Dla grafu nieskierowanego wykorzystuje Twierdzenie Orego (suma stopni par niesąsiadujących $\geq n$). Dla skierowanego Twierdzenie Woodalla.
- 2.3. **SEC (Przeszukiwania – Euler):** Klasyczny backtracking operujący na krawędziach. Wędruje po grafie "spalając" za sobą krawędzie, a przy ślepych zaułku cofa się (backtrack) odtwarzając krawędź.
- 2.4. **SHC (Przeszukiwania – Hamilton):** Metoda Roberta-Floresa (backtracking operujący na wierzchołkach).

3. Metodologia testów i pomiarów - eksperymenty podzielono na dwie procedury pomiarowe:

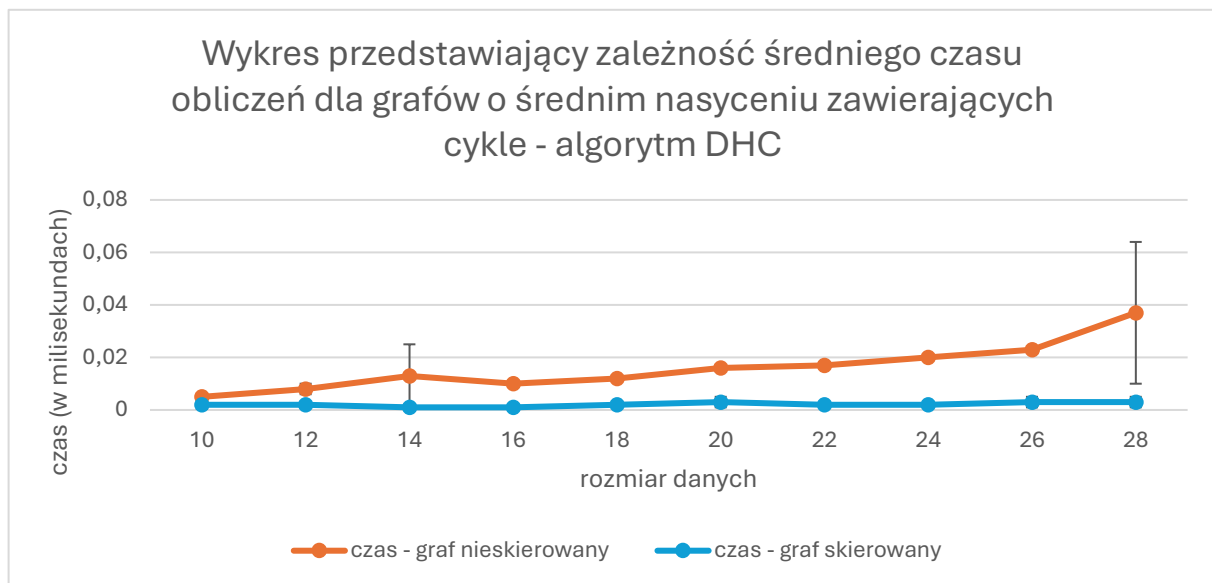
- 3.1. **Procedura 1 (Grafy losowe):** Dla wszystkich 4 algorytmów badano grafy o parametrach $s = \{10, 20, 30, 40, 50, 60, 70, 80, 90\}\%$. Wygenerowano po 10 niezależnych grafów, z których nie każdy musiał posiadać cykl.
- 3.2. **Procedura 2 (Grafy wymuszone):** Skupiono się tylko na algorytmach SEC i SHC dla $s = \{50, 60, 70, 80, 90\}\%$. Specjalne generatory tworzyły grafy gwarantujące istnienie cyklu eulerowskiego lub hamiltonowskiego.

Z powodu NP-trudności problemu Hamiltona dobrano dwa przedziały wielkości: dla Eulera $n = \{100, 200, \dots, 1000\}$, a dla Hamiltona $n = \{10, 12, \dots, 28\}$. Wyniki uśredniono i wyliczono odchylenie standardowe. Czas został zmierzony w milisekundach. Dodatkowo zaimplementowano bezpiecznik zapobiegający przepełnieniu stosu (Stack Overflow) przy bardzo głębokiej rekurencji SEC.

4. Analiza wykresów

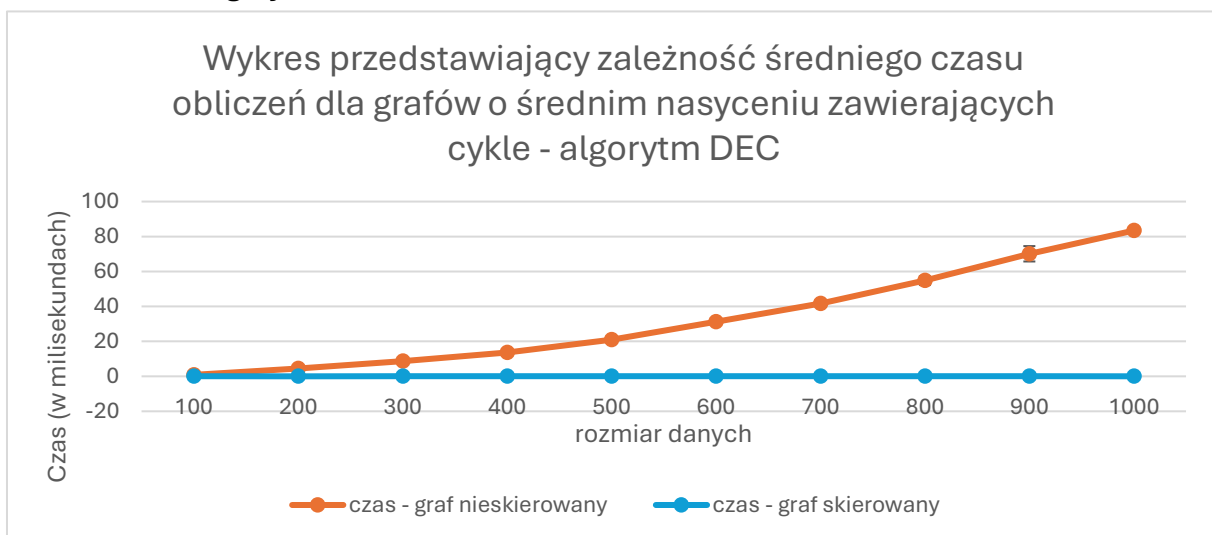
4.1. Wykresy liniowe przedstawiające zależność średniego czasu obliczeń t (oś Y) od liczby wierzchołków n (oś X) dla grafów o średnim nasyceniu ($s = 50\%$) zawierających cykle.

4.1.1. Algorytm DHC



Wykres pokazuje czas weryfikacji twierdzeń z wymuszonym cyklem Hamiltona. Czas działania dla obu reprezentacji jest marginalny i wynosi ułamki milisekund. Krzywa dla grafów nieskierowanych rośnie nieznacznie szybciej, co wynika z konieczności weryfikacji sumy stopni par wierzchołków niesąsiadujących, podczas gdy dla grafów skierowanych badane są po prostu odpowiednie stopnie bezpośrednio z macierzy grafu. Niewielki wzrost odchylenia standardowego na końcu wykresu wynika z naturalnych fluktuacji procesora przy tak krótkich pomiarach.

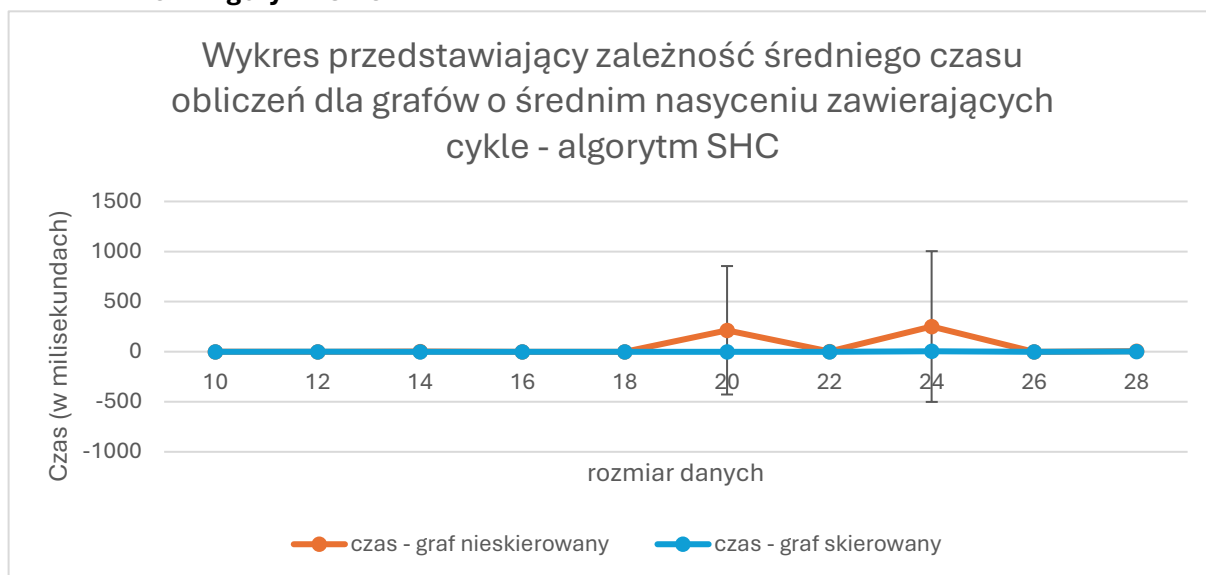
4.1.2. Algorytm DEC



Zależność czasu weryfikacji warunków cyklu Eulera silnie zależy od reprezentacji. Dla grafów nieskierowanych czas rośnie wielomianowo. Ponieważ graf ma gwarantowany cykl (wszystkie stopnie są parzyste), algorytm zawsze musi w całości wykonać drugą część testu – przejście BFS sprawdzające spójność grafu, co kosztuje najwięcej czasu. Z kolei dla grafów skierowanych czas oscyluje wokół zera – wynika to z faktu, że struktura macierzy grafu pozwala na błyskawiczne

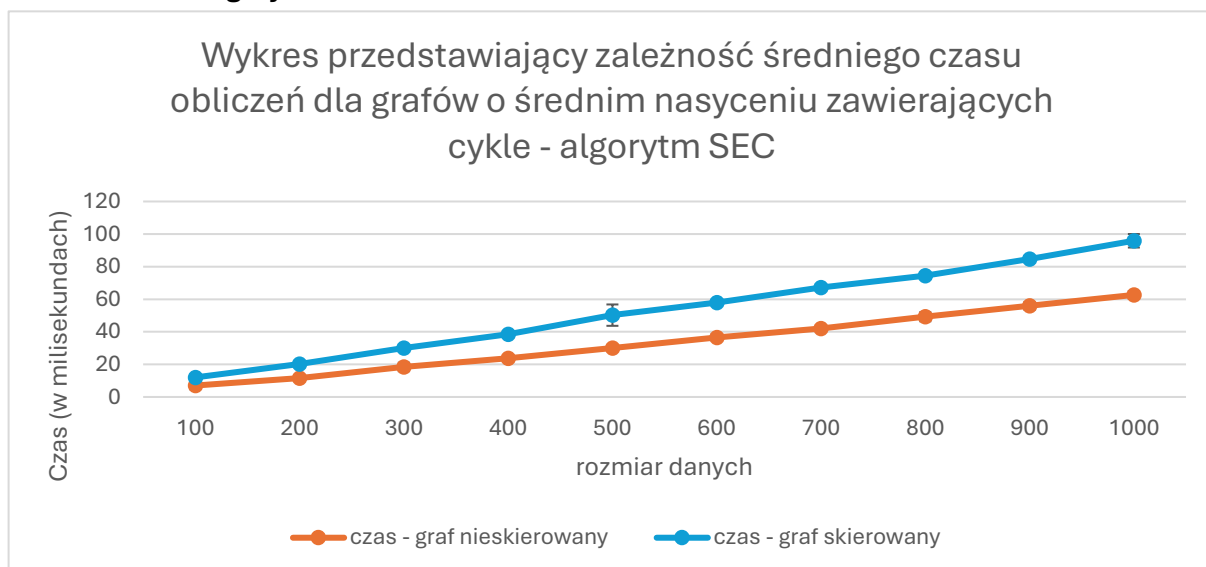
zliczanie stopni wchodzących i wychodzących, a badanie spójności na tej strukturze jest dla tych danych bardzo wydajne.

4.1.3. Algorytm SHC



Wykres poszukiwania cyklu Hamiltona (metoda Roberta-Floresa) dla nasycenia 50% ukazuje czasy bliskie zeru. Ponieważ grafy w tej próbie mają wymuszony, wbudowany cykl, a nasycenie 50% zapewnia dużą gęstość krawędzi, algorytm DFS (przeszukiwanie w głąb) odnajduje poprawną ścieżkę niemal natychmiast – w pierwszej gałęzi drzewa wywołań. Wyraźne skoki na słupkach odchylenia (np. dla grafu nieskierowanego przy $N=20$ i $N=24$) oznaczają, że w niektórych pojedynczych próbach algorytm niefortunnie wpadł w ślepy zaułek i musiał wykonać serię backtracków (cofań), co wygenerowało lokalne piki czasu.

4.1.4. Algorytm SEC

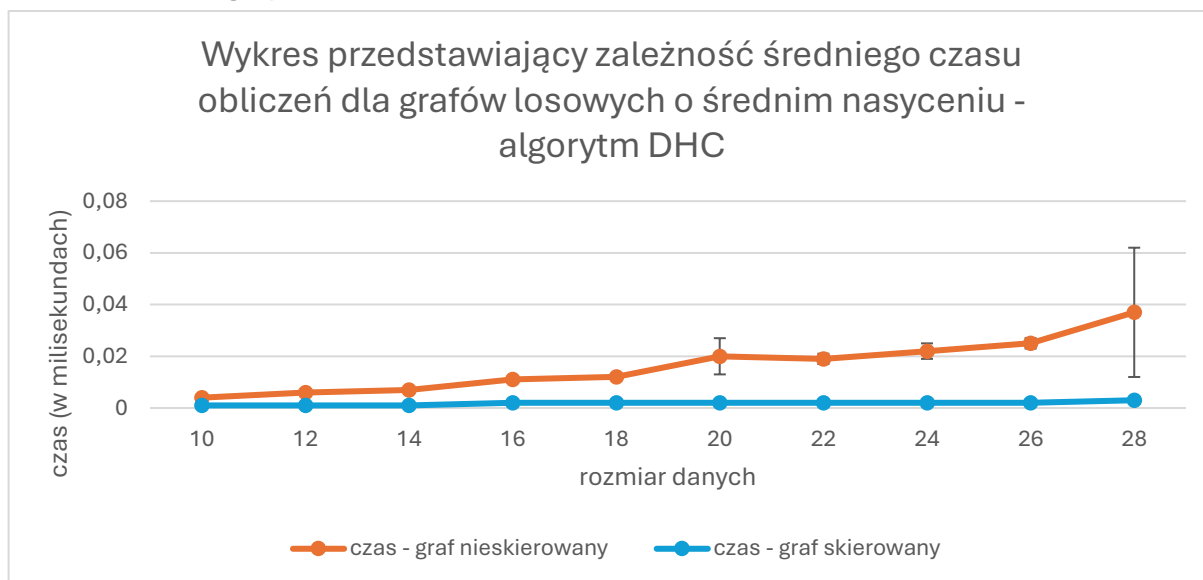


W przypadku poszukiwania cyklu Eulera czas rośnie stabilnie i liniowo w stosunku do powiększającego się rozmiaru danych. Dla $N=1000$ poszukiwania na grafie skierowanym kończą się po ok. 100 ms, a na nieskierowanym po ok. 60 ms. Skoro istnienie cyklu jest gwarantowane generatorem, algorytm sukcesywnie "zdejmuje" kolejne krawędzie z grafu i dodaje do ścieżki.

Dłuższy czas dla grafu skierowanego wynika z narzutu pamięciowego macierzy grafu (konieczność modyfikowania "wskaźników" wielokrotnych list) w porównaniu do prostego zerowania komórek w macierzy sąsiedztwa dla grafu nieskierowanego.

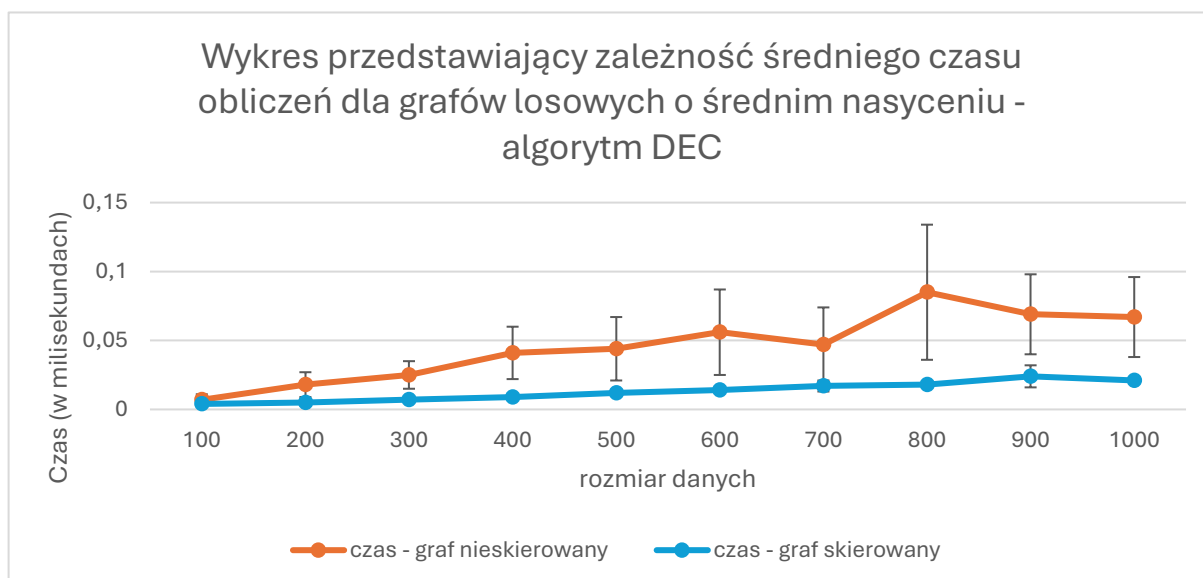
4.2. Wykresy liniowe przedstawiające zależność średniego czasu obliczeń t (oś Y) od liczby wierzchołków n (oś X) dla grafów o średnim nasyceniu ($s = 50\%$), które mogą zawierać lub nie zawierać cykli.

4.2.1. Algorytm DHC



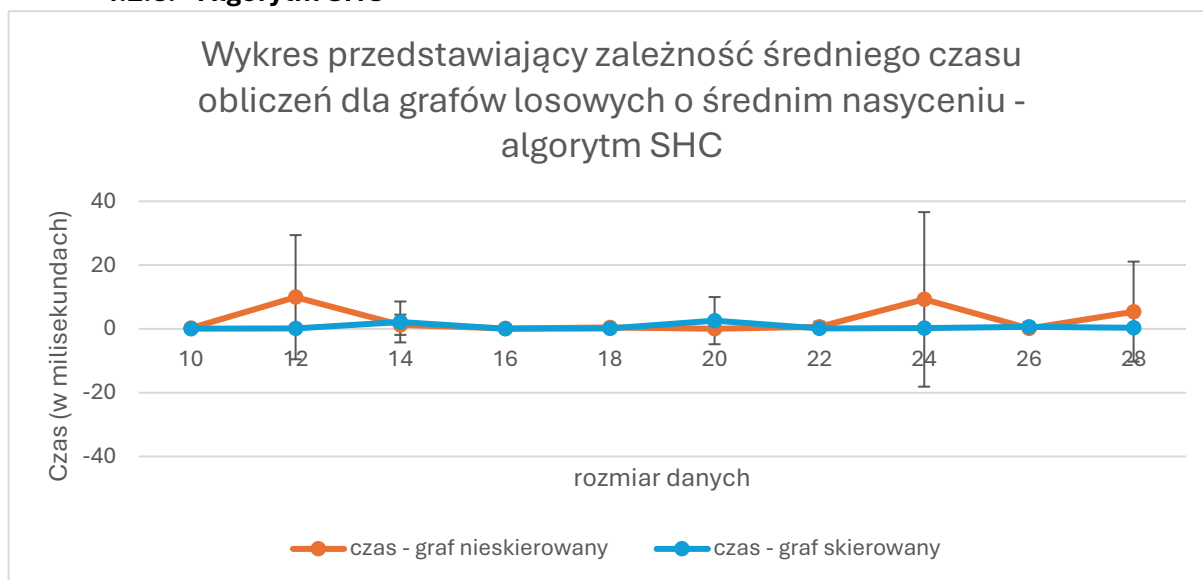
Dla grafów losowych krzywa decyzyjnego weryfikatora Hamiltona przebiega niemal identycznie jak w punkcie 4.1.1 (z maksimum ok. 0,04 ms dla nieskierowanych). Udowadnia to, że algorytm ten ma ściśle zdefiniowaną i deterministyczną złożoność obliczeniową zależną wyłącznie od liczby wierzchołków N . Weryfikacja warunków z twierdzeń polega na szybkim sumowaniu stopni i niezależnie od istnienia cyklu algorytm zawsze wykonuje podobną, stałą liczbę operacji.

4.2.2. Algorytm DEC



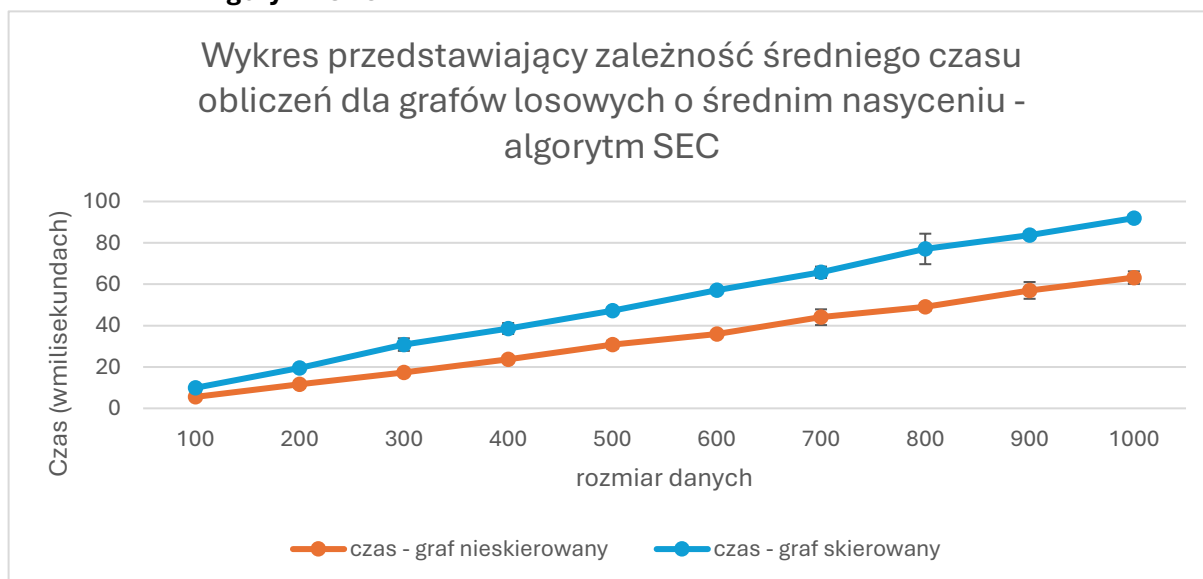
Na tym wykresie widać drastyczną zmianę względem weryfikacji grafów eulerowskich z punktu 4.1.2. Czas dla grafu nieskierowanego zmalał stukrotnie (z 80 ms na ok. 0,08 ms). W losowym grafie szansa na to, że wszystkie wierzchołki będą miały parzysty stopień, jest znikoma. Dlatego w pierwszej pętli algorytm błyskawicznie napotyka wierzchołki o nieparzystym stopniu, od razu przerywa działanie zwracając "Fałsz" i całkowicie pomija kosztowny krok sprawdzania spójności (BFS). Dzięki temu algorytm odrzuca grafy losowe w ułamku milisekundy.

4.2.3. Algorytm SHC



Wykres czasu przeszukiwania Hamiltona dla losowych grafów jest bardzo płaski, jednak zauważalne są liczne wychylenia (np. szczyty i duże odchylenia dla $N=12$ oraz $N=24$ dla grafu nieskierowanego). Kształt ten wynika z wysokiej losowości procedury. W wielu próbach algorytm w ogóle nie odnajdywał cyklu lub ścieżka okazywała się zablokowana już na wczesnym etapie, co skutkowało szybkim powrotem. Skoki oznaczają pojedyncze przypadki, w których algorytm przed ostatecznym zwróceniem wyniku negatywnego zdążył "zabłądzić" w skomplikowanym podgrafie, generując kilkadziesiąt backtracków.

4.2.4. Algorytm SEC

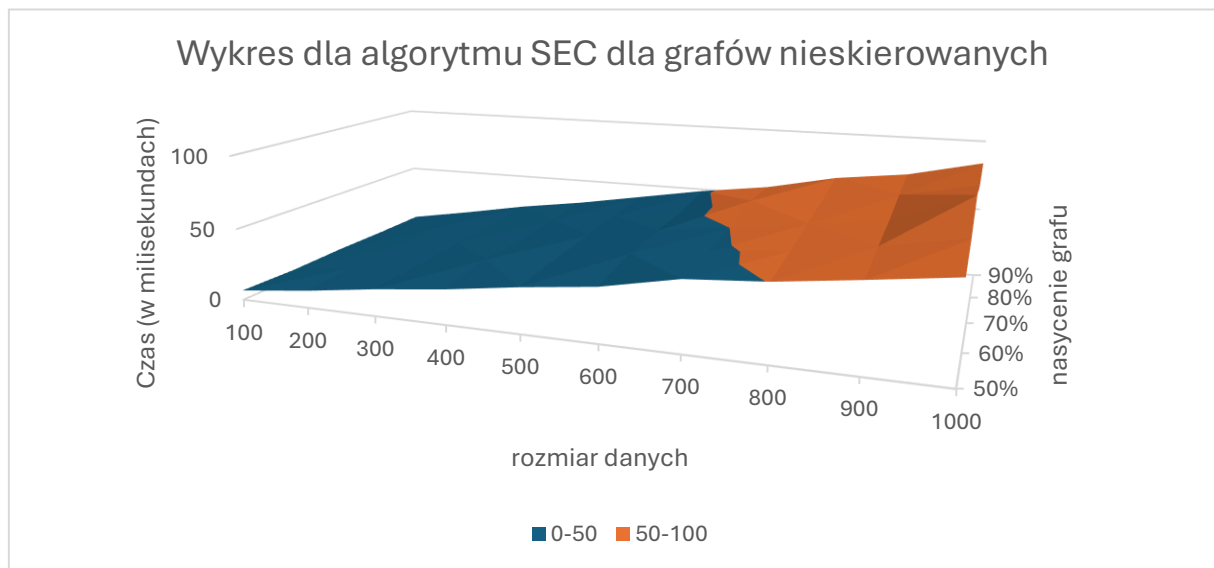


Działanie naiwnego algorytmu powracania dla cyklu Eulera w grafach losowych wygenerowało wykres niezwykle zbliżony do tego z punktu 4.1.4 (liniowy wzrost, czas max 100 ms dla skierowanych). W grafach losowych (bez wymuszonego cyklu) algorytm próbuje eksplorować drzewo wszystkich możliwości ułożenia krawędzi, które jest gigantyczne. Wyniki te wskazują, że zastosowany w kodzie ogranicznik bezpieczeństwa (limit głębokości stosu/backtracków) ucina poszukiwania na ustalonym pułapie operacji, chroniąc program przed zawieszeniem i dając przewidywalny, liniowy czas wykonania próby "spalenia na panewce".

4.3. Wykresy powierzchniowe 3D $t=f(n,s)$ zależności czasu obliczeń t od liczby n wierzchołków w grafie i nasycenia s .

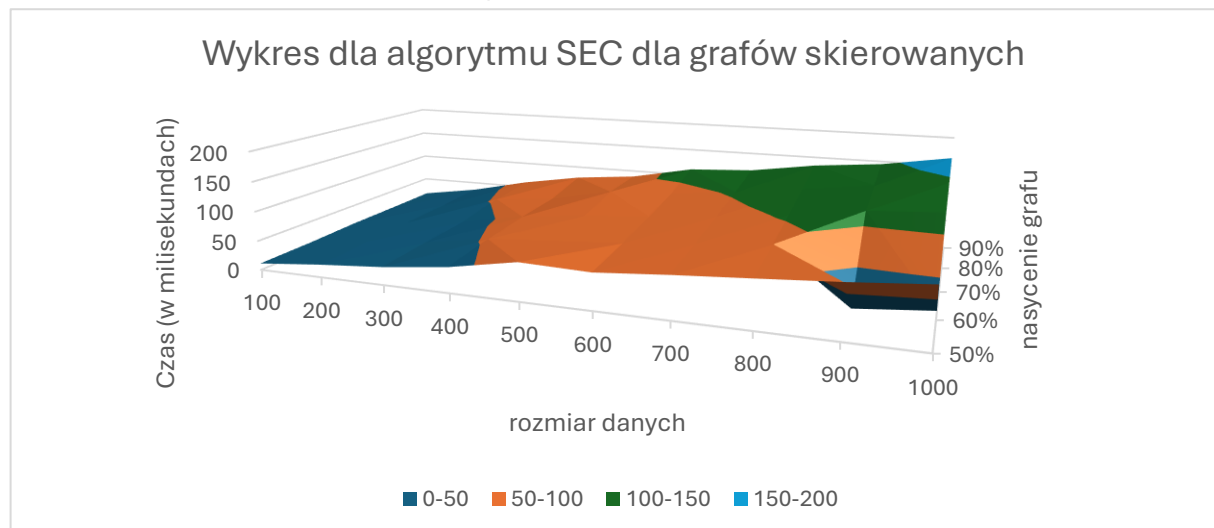
4.3.1. Algorytm SEC

4.3.1.1. Graf nieskierowany



Wykres powierzchniowy udowadnia, że czas poszukiwania cyklu Eulera na macierzy sąsiedztwa rośnie kaskadowo w miarę jednoczesnego zwiększania liczby wierzchołków (N) oraz nasycenia grafu. Powierzchnia osiąga swój najwyższy szczyt w prawym krańcu (dla $N=1000$ oraz $S=90\%$). Ponieważ rozwiązanie problemu Eulera wymaga fizycznego przejścia przez wszystkie istniejące krawędzie, im graf jest gęstszy (wyższe S), tym trasa jest dłuższa, co bezpośrednio przekłada się na proporcjonalny, liniowy wzrost czasu procesora.

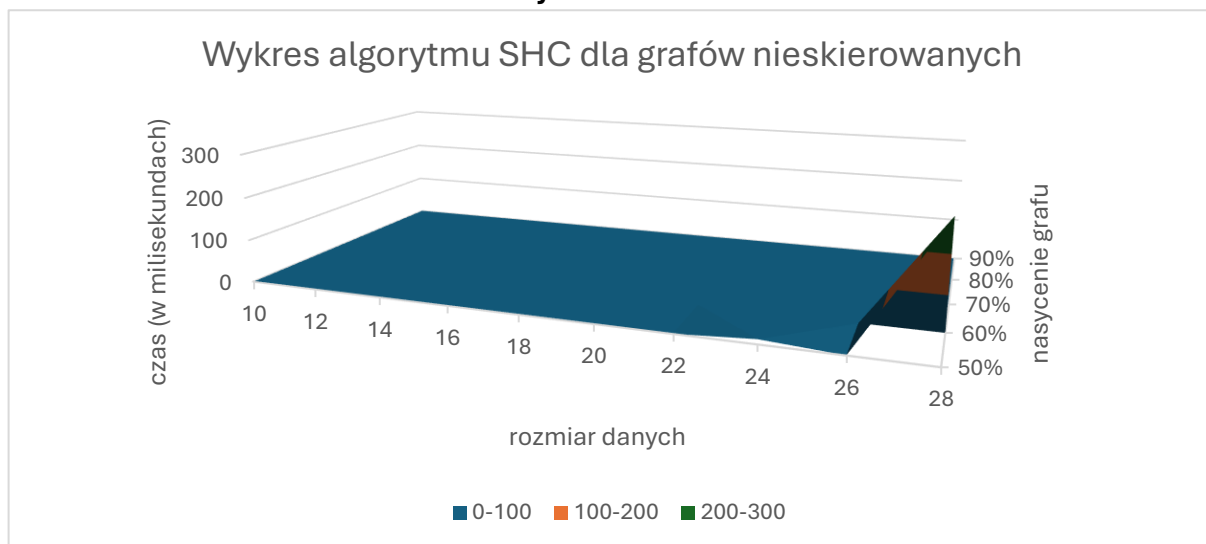
4.3.1.2. Graf skierowany



Powierzchnia 3D dla grafu skierowanego (opartego na macierzy grafu) ukazuje analogiczny trend rosnący względem N i S. Płaszczyzna jest tutaj nieco bardziej popalowana – dostrzegamy gwałtowne wzniesienia przy nasyceniu 60% i 90%. Nieregularności te w dużej mierze wynikają z kosztów operowania na listach zagnieżdżonych w reprezentacji macierzy grafu (odpinanie wskaźników następników po "wejściu" w krawędź łuku), co przy określonych, gęstych układach staje się zauważalnie bardziej czasochłonne niż w prostych strukturach nieskierowanych.

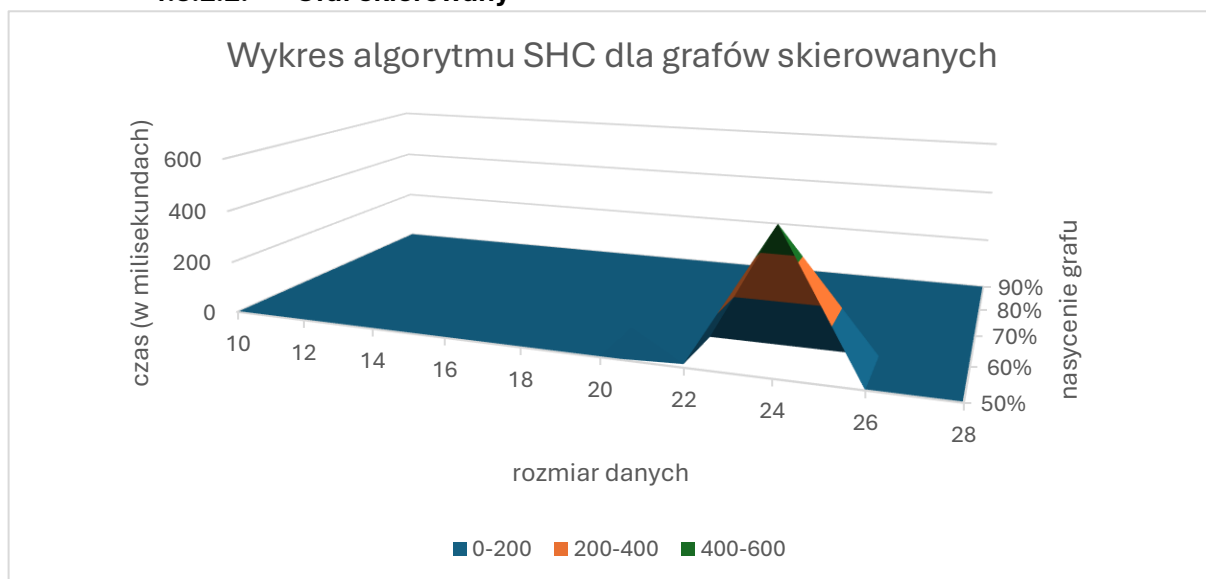
4.3.2. Algorytm SHC

4.3.2.1. Graf nieskierowany



Wykres prezentuje niezwykle ciekawe zachowanie backtrackingu w problemach NP-trudnych. Płaszczyzna jest niemal całkowicie płaska (czas bliski zera) dla większości parametrów, jednak w okolicach wysokiego nasycenia (70-90%) dla największego rozmiaru $N=28$ pojawia się gwałtowny, stromy "kolec" (wzrost czasu do ponad 200 ms). Udowadnia to występowanie lokalnych "eksplozji kombinatorycznych". W zdecydowanej większości przypadków DFS błyskawicznie "zgaduje" dobrą ścieżkę, ale dla konkretnych, losowo wygenerowanych w tej próbie układów krawędzi, algorytm zapętlał się w fałszywych rozgałęzieniach, wykonując setki tysięcy cofnięć.

4.3.2.2. Graf skierowany



Powierzchnia dla problemu Hamiltona na grafach skierowanych w 100% potwierdza losową naturę heurystyki z powracaniem. Na płaskim wykresie wyrasta samotny, bardzo wysoki szczyt w środku badanego zakresu (dla $N=24$ i $S=70\%$). Ten nagły wzrost opóźnienia potwierdza, że dla problemu Hamiltona, w przeciwieństwie do Eulera, czas działania nie rośnie stabilnie i proporcjonalnie. Zależy on w głównej mierze od topologii konkretnej instancji grafu – pojedynczy, wyjątkowo złośliwy układ skierowanych łuków może "uwięzić" algorytm DFS, zmuszając go do drastycznie dłuższego poszukiwania rozwiązania.

5. Analiza wyników eksperymentów obliczeniowych:

5.1. Jak różni się liczba backtracków w obu problemach? Dlaczego zastosowanie tej samej techniki (backtracking) prowadzi do tak różnych wyników dla problemu Eulera i Hamiltona?

Problem Eulera (SEC): Liczba nawrotów jest zazwyczaj bliska zeru lub bardzo mała. Jeśli graf posiada cykl Eulera, algorytmy (zmodyfikowany DFS z usuwaniem krawędzi) bardzo rzadko "ślepną".

Problem Hamiltona (SHC): Liczba nawrotów rośnie lawinowo i osiąga gigantyczne wartości (nawet miliony dla stosunkowo małych grafów).

Dlaczego tak jest? Problem Eulera opiera się na **własnościach lokalnych**. Jeśli sprawdziliśmy, że stopnie wierzchołków są parzyste, mamy matematyczną gwarancję, że nie utkniemy na stałe (zawsze będzie krawędź do wyjścia, z wyjątkiem powrotu do startu). Z kolei Cykl Hamiltona opiera się na **własnościach globalnych**. Decyzja podjęta w kroku 1 może ujawnić swój fatalny skutek dopiero w kroku 20, zmuszając algorytm do wycofania się o 19 kroków.

5.2. Jak zmienia się czas działania wraz ze wzrostem n ?

Euler: Czas działania rośnie **wielomianowo** (liniowo lub kwadratowo, np. $O(V+E)$ lub $O(V^2)$, zależnie od implementacji). Wzrost jest łagodny – podwojenie liczby wierzchołków n spowoduje zaledwie kilkukrotny wzrost czasu. Jednakże w tej implementacji złożoność ta wynosi $O(n^2)$. Wynika to bezpośrednio z wymogów zadania i narzuconej reprezentacji grafu – ponieważ algorytm operuje na macierzy sąsiedztwa/grafu, przy każdym kroku przeszukiwania w głąb musi on iterować po n elementach wiersza macierzy, aby odnaleźć dostępne krawędzie. Mimo to, czas ten wciąż rośnie wielomianowo, co potwierdziły eksperymenty (czasy rzędu ułamków milisekund nawet dla większych grafów).

Hamilton: Czas działania rośnie **wykładniczo/silniowo** (w najgorszym razie $O(n!)$). Podwojenie liczby n (np. z 10 na 20) sprawi, że czas wykonania skoczy z milisekund do lat.

5.3. Jak zmiana nasycenia grafu wpływa na liczbę operacji cofania w algorytmie SHC? Czy łatwiej znaleźć cykl Hamiltona w grafie rzadkim czy gęstym?

Zdecydowanie **łatwiej znaleźć cykl Hamiltona w grafie gęstym**.

- W grafach o dużym nasyceniu (bliskich grafom pełnym) istnieje tak wiele alternatywnych ścieżek, że algorytm DFS bardzo szybko "wstrzeliwuje się" w poprawną trasę. Liczba backtracków jest wtedy bardzo mała.
- W grafach **bardzo rzadkich** algorytm też działa szybko, bo szybko "odbija się od ściany" (szybko udowadnia, że cyklu nie ma, np. trafiając na wierzchołek o stopniu 1).

- **Najgorsze są grafy o średnim nasyceniu.** Algorytm ma wystarczająco dużo ścieżek, by błądzić godzinami w potężnym drzewie przeszukiwań, ale wystarczająco mało, by łatwo znaleźć cykl nie było proste. Wtedy liczba backtracków rośnie do gigantycznych rozmiarów.

5.4. Dla jakich typów grafów (lub parametrów n , s) algorytmy z powracaniem wykazywały największe odchylenie standardowe czasu obliczeń?

Największe odchylenie standardowe czasu obliczeń występuje w algorytmie **SHC dla grafów o średnim nasyceniu**. W tych grafach algorytm z powracaniem bywa niezwykle wrażliwy na to, **którego sąsiada wybierze jako pierwszego**. Z powodu czystego szczęścia może znaleźć cykl w ułamku sekundy (0 nawrotów), a przy innym wylosowaniu tego samego grafu – szukać go przez 10 minut w złej gałęzi drzewa decyzyjnego. To sprawia, że wyniki mocno "skaczą".

5.5. Czy problem Eulera rzeczywiście jest łatwy w praktyce?

Tak, jest wybitnie łatwy. Ponieważ należy do klasy P (problemów rozwiązywalnych w czasie wielomianowym), w praktyce możemy go rozwiązywać dla grafów posiadających miliony wierzchołków i krawędzi w ułamki sekund. W projekcie zastosowano algorytm oparty na przeszukiwaniu w głąb (DFS) z usuwaniem krawędzi i mechanizmem powrotów (backtrackingiem). Dzięki specyfice cyklu Eulera (gdzie większość dróg prowadzi do poprawnego rozwiązania), algorytm wykonuje bardzo mało nawrotów, co pozwala na błyskawiczne działanie w praktyce dla testowanych rozmiarów grafów.

5.6. W którym momencie algorytm Hamiltona przestaje być wykonywalny?

Dla standardowej metody backtrackingu (bez dodatkowych optymalizacji) algorytm traci wydajność zazwyczaj już w okolicach **$n = 15$ do $n=25$** wierzchołków (zależnie od nasycenia). Drzewo przeszukiwań staje się po prostu zbyt ogromne. W pewnym momencie czas wykonania z milisekund nagle staje się tak długi, że należy ręcznie przerywać działanie programu.

5.7. Czy zauważasz różnicę w czasie poszukiwania rozwiązania w grafach zawierających cykle i grafach, które mogą ich nie zawierać?

Graf z cyklem: Algorytm przerywa działanie w momencie znalezienia pierwszego rozwiązania (early exit). Może to nastąpić bardzo wcześnie.

Graf bez cyklu: Algorytm musi sprawdzić **absolutnie każdą z możliwych dróg**, aby ze 100% pewnością orzec, że cyklu nie ma. Dla grafu bez cyklu SHC zaprezentuje swój najgorszy możliwy czas i maksymalną liczbę nawrotów. Czas zawsze będzie drastycznie dłuższy.

5.8. Czy narzucona reprezentacja maszynowa grafu ułatwiała czy utrudniała implementację konkretnych algorytmów (np. sprawdzanie spójności lub wyszukiwanie sąsiadów)?

Macierz sąsiedztwa (nieskierowane): Jest bardzo prosta i intuicyjna w implementacji (zwykła tablica 2D). Ułatwia sprawdzanie spójności, ale spowalnia szukanie sąsiadów w grafach rzadkich (trzeba iterować przez cały wiersz pełen zer w czasie $O(n)$).

Macierz grafu (skierowane): Implementacja jest trudna, podatna na błędy (wskaźniki, indeksowanie z przesunięciami). Jednak drastycznie przyspiesza wyszukiwanie sąsiadów, ponieważ pozwala "przeskakiwać" od razu po istniejących krawędziach, ignorując puste miejsca. W zastosowaniach z dużą liczbą backtracków, ten zysk na optymalizacji pętli jest bardzo ważny.

6. Analiza złożoności obliczeniowej

6.1. Tabelka ze złożonością obliczeniową

Problem i metoda	Teoretyczna klasa złożoności	Złożoność zastosowanego algorytmu	Wyjaśnienie implementacji
Euler – decyzyjny (DEC)	$O(n+m)$	$O(n^2)$	Zliczanie stopni wierzchołków i sprawdzanie spójności (BFS) wymaga przeszukania całej macierzy $n \times n$.
Euler – przeszukiwanie (SEC)	$O(n+m)$	$O(n^2)$	Przeszukiwanie (np. modyfikacja algorytmu DFS) z usuwaniem krawędzi w macierzy.
Hamilton – decyzyjny (DHC)	NP-zupełny	$O(n^3)$	Sprawdzanie warunków wystarczających (Orego/Woodalla) dla par wierzchołków niepołączonych krawędzią.
Hamilton – przeszukiwanie (SHC)	NP-zupełny	$O(n!)$	Algorytm z powrotami (backtracking) w najgorszym przypadku sprawdza wszystkie permutacje wierzchołków.

6.2. Różnica między klasą złożoności problemu a złożonością rozwiązującego go algorytmu?

- **Złożoność problemu (Klasa złożoności):** Określa matematyczną trudność samego zadania. Wskazuje na absolutne minimum czasu lub pamięci, jakiego wymagałby najlepszy z możliwych algorytmów, aby dany problem rozwiązać. Problem jest przypisywany do klas takich jak **P** (łatwe) czy **NP-trudne** (bardzo trudne).
- **Złożoność algorytmu:** Określa czas działania i zużycie pamięci dla konkretnego, napisanego przez programistę kodu.

Wniosek: Złożoność algorytmu może być gorsza niż złożoność problemu (np. można użyć powolnego algorytmu o złożoności $O(n^3)$ do rozwiązania łatwego problemu z klasy P). Jednak nie jest możliwe napisanie algorytmu wielomianowego (szybkiego) dla problemu z klasy NP-trudnych.

6.3. Dlaczego problem cyklu Eulera należy do klasy P?

Klasa **P** (Polynomial) grupuje problemy, które potrafimy rozwiązać w czasie wielomianowym (np. $O(n)$, $O(n^2)$). Problem cyklu Eulera należy do tej klasy, ponieważ jego rozwiązanie **nie wymaga przeszukiwania w głąb ani sprawdzania wielu kombinacji ścieżek**.

Leonhard Euler udowodnił, że istnienie cyklu zależy wyłącznie od **lokalnych własności wierzchołków**. Wystarczy zweryfikować dwa warunki:

1. Czy graf jest spójny (algorytm BFS/DFS w czasie wielomianowym).
2. Czy każdy wierzchołek ma parzysty stopień (dla grafów nieskierowanych) lub zbilansowany stopień wejściowy i wyjściowy (dla skierowanych).

Sprowadza się to do prostego zliczenia wartości w macierzy, co z definicji jest operacją szybką i deterministyczną.

6.4. Dlaczego problem cyklu Hamiltona jest silnie NP-trudny/NP-zupełny?

Aby problem należał do klasy **NP-zupełnych**, musi spełniać dwa warunki, które problem Hamiltona spełnia idealnie:

1. **Należy do klasy NP:** Jeśli otrzymamy gotową odpowiedź (np. ciąg wierzchołków), potrafimy w szybkim czasie (wielomianowym) zweryfikować, czy jest to prawidłowy cykl Hamiltona.
2. **Jest NP-trudny:** Nie istnieje żadna znana matematyczna właściwość lokalna (jak parzystość stopni w problemie Eulera), która gwarantowałaby istnienie cyklu Hamiltona. Wymusza to analizowanie struktury globalnie.

Standardowe algorytmy (jak backtracking) muszą w pesymistycznym przypadku przeszukać całe drzewo decyzyjne, sprawdzając wszystkie możliwe permutacje dróg. Przestrzeń poszukiwań rośnie w tempie silniowym – $O(n!)$ – co czyni problem nierozwiązywalnym w rozsądnym czasie nawet dla stosunkowo niewielkich grafów (np. $n > 20$).

6.5. Czy istnieją grafy hamiltonowskie, które nie spełniają twierdzenia Ore'go/Woodalla?

Tak, istnieje ich bardzo wiele. Twierdzenia Ore'go i Woodalla definiują wyłącznie **warunki wystarczające, ale nie konieczne**.

Oznacza to, że:

- Jeśli graf spełnia warunek z twierdzenia to na pewno posiada cykl Hamiltona.
- Jeśli graf **nie spełnia** tego warunku to **nadal może posiadać cykl Hamiltona** (twierdzenie po prostu nie potrafi go wykryć).

Przykład:

Rozważmy graf będący zwykłym, pustym w środku cyklem o 5 wierzchołkach (pięciokąt).

- Jest to z definicji graf hamiltonowski, ponieważ obejście jego obwodu tworzy cykl Hamiltona.
- W grafie C_5 każdy wierzchołek ma stopień równy 2. Jeśli weźmiemy dwa dowolne wierzchołki niepołączone krawędzią, suma ich stopni wynosi $2 + 2 = 4$. Zgodnie z twierdzeniem Ore'go, ta suma musiałaby być większa lub równa liczbie wszystkich wierzchołków ($4 \geq 5$), co jest fałszem. Twierdzenie Ore'go nie potwierdza obecności cyklu, chociaż on tam jest.

7. Wnioski końcowe

Porównanie problemów i ich praktyczna wykonalność

- Przeprowadzone badania wyraźnie uwydatniają fundamentalną różnicę między problemem łatwym obliczeniowo (należącym do klasy P) a problemem NP-zupełnym.
- **Cykl Eulera** opiera się na lokalnych właściwościach wierzchołków (np. parzystości stopni). Dzięki temu algorytmy go poszukujące są wysoce skalowalne. W praktyce wyznaczenie tego cyklu jest bezproblemowo wykonalne nawet dla bardzo dużych struktur danych w czasie rzędu ułamków sekund.
- **Cykl Hamiltona** wymaga analizy globalnej i nie posiada w pełni skutecznych, łatwych do weryfikacji warunków lokalnych. Praktyczna wykonalność poszukiwania tego cyklu metodami dokładnymi kończy się zaledwie na kilkudziesięciu wierzchołkach, po czym następuje całkowita utrata wydajności ze względu na zbyt długi czas obliczeń.

Wpływ zastosowanego podejścia (backtracking)

- Mimo zastosowania tej samej techniki z powrotami do obu problemów, daje ona drastycznie różne rezultaty.
- W **wersji przeszukiwania dla problemu Eulera (SEC)**, algorytm wykorzystujący strategię usuwania odwiedzonych krawędzi działa deterministycznie. Z powodu gwarancji istnienia cyklu przy spełnieniu odpowiednich warunków początkowych, liczba nawrotów jest znikoma, a czas rośnie w sposób wielomianowy.
- W **wersji przeszukiwania dla problemu Hamiltona (SHC)**, klasyczny backtracking bez dodatkowych modyfikacji wywołuje zjawisko eksplozji kombinatorycznej o złożoności silniowej $O(n!)$.
- Dla grafów o średnim nasyceniu wymusza to wykonywanie milionów powrotów ze ślepych zaułków, całkowicie paraliżując działanie programu.

Efektywność reprezentacji maszynowej grafu

- Macierz sąsiedztwa (dla grafów nieskierowanych): Zwykła macierz 2D okazała się w pełni wystarczająca i bardzo intuicyjna w implementacji. Zapewniała błyskawiczny czas sprawdzania istnienia krawędzi (czas stały $O(1)$) oraz ułatwiała pisanie algorytmów weryfikujących spójność grafu (co było kluczowe dla problemu Eulera). Główną wadą tego rozwiązania był fakt, że w przypadku grafów rzadkich zmuszała ona algorytmy do nieefektywnego iterowania po pustych komórkach wiersza w czasie liniowym $O(n)$, co w przypadku algorytmu SHC niepotrzebnie obciążało procesor.

- Macierz grafu (dla grafów skierowanych): Specyficzna struktura tej macierzy, rozszerzona o bezpośrednią listę następników, sprawdzała się znakomicie pod kątem optymalizacji. Mimo wyższego stopnia skomplikowania samego kodu (co utrudniało początkową implementację), pozwalała ona na natychmiastowe przeskakiwanie do rzeczywistych sąsiadów wierzchołka. W przypadku algorytmów o drastycznie wysokiej liczbie nawrotów, takich jak przeszukiwanie cyklu Hamiltona (SHC), taka minimalizacja zbędnych iteracji po zerach stanowiła kluczową przewagę optymalizacyjną.

Kompromis czas vs pamięć

- Wykorzystanie wszelkich reprezentacji opartych na strukturach macierzowych faworyzuje **szybkość dostępu** kosztem **wysokiego zużycia pamięci**.
- Odczyt i modyfikacja krawędzi odbywa się w optymalnym czasie stałym $O(1)$, jednak przestrzeń pamięciowa rośnie kwadratowo ($O(n^2)$).
- W przypadku analizy grafów o dużym rozmiarze, lecz bardzo niskim nasyceniu krawędziami (grafy rzadkie), takie podejście prowadzi do marnotrawstwa zasobów pamięciowych na przechowywanie wartości zerowych.

8. Tabelki pomiarów:

8.1. Tabelki przedstawiające zależność średniego czasu obliczeń od liczby wierzchołków n dla grafów o średnim nasyceniu ($s = 50\%$) zawierających cykle.

8.1.1. Algorytm DHC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
10	0,005	0,001	0,002	0
12	0,008	0,002	0,002	0
14	0,013	0,012	0,001	0
16	0,01	0	0,001	0
18	0,012	0,001	0,002	0
20	0,016	0,001	0,003	0
22	0,017	0,001	0,002	0
24	0,02	0	0,002	0
26	0,023	0,001	0,003	0
28	0,037	0,027	0,003	0

8.1.2. Algorytm DEC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
100	0,867	0,028	0,006	0,011
200	4,533	0,805	0,004	0,001
300	8,627	0,811	0,007	0,001
400	13,693	0,825	0,01	0,001
500	20,967	0,742	0,012	0,002
600	31,165	0,914	0,013	0,001
700	41,74	1,576	0,015	0,001
800	54,773	2,533	0,016	0

900	70,118	4,402	0,018	0,001
1000	83,395	0,899	0,02	0,001

8.1.3. Algorytm SHC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
10	0,033	0,022	0,053	0,06
12	0,043	0,064	0,038	0,02
14	0,38	0,884	0,139	0,13
16	0,102	0,175	0,056	0,03
18	0,036	0,038	0,126	0,17
20	213,826	641,34	0,182	0,21
22	1,721	3,342	0,112	0,1
24	251,156	752,685	3,905	10,9
26	0,265	0,321	0,183	0,14
28	3,85	10,908	0,859	2,24

8.1.4. Algorytm SEC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
100	7,022	2,294	12,001	1,293
200	11,669	1,261	20,279	1,206
300	18,446	2,632	30,02	0,265
400	23,857	1,761	38,502	0,6
500	30,12	1,542	50,235	6,551
600	36,635	2,082	57,938	0,987
700	41,969	1,465	67,175	1,489
800	49,244	2,785	74,383	1,572
900	55,932	2,394	84,703	1,828
1000	62,637	2,424	95,894	4,148

8.2. Tabelki przedstawiające zależność średniego czasu obliczeń od liczby wierzchołków n dla grafów o średnim nasyceniu ($s = 50\%$), które mogą zawierać lub nie zawierać cykli.

8.2.1. Algorytm DHC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
10	0,004	0	0,001	0,001
12	0,006	0,001	0,001	0,001
14	0,007	0	0,001	0,001
16	0,011	0,001	0,002	0,001
18	0,012	0,001	0,002	0,001
20	0,02	0,007	0,002	0,001
22	0,019	0,002	0,002	0,001

24	0,022	0,003	0,002	0,001
26	0,025	0,002	0,002	0,001
28	0,037	0,025	0,003	0,001

8.2.2. Algorytm DEC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
100	0,007	0,004	0,004	0,001
200	0,018	0,009	0,005	0,001
300	0,025	0,01	0,007	0,001
400	0,041	0,019	0,009	0
500	0,044	0,023	0,012	0,002
600	0,056	0,031	0,014	0,002
700	0,047	0,027	0,017	0,004
800	0,085	0,049	0,018	0,002
900	0,069	0,029	0,024	0,008
1000	0,067	0,029	0,021	0,001

8.2.3. Algorytm SHC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
10	0,213	0,5	0,069	0,128
12	9,932	19,469	0,174	0,276
14	1,289	3,169	2,169	6,41
16	0,109	0,15	0,08	0,061
18	0,444	0,423	0,176	0,291
20	0,041	0,046	2,577	7,397
22	0,68	1,569	0,117	0,062
24	9,242	27,366	0,208	0,241
26	0,173	0,374	0,644	1,478
28	5,405	15,684	0,31	0,385

8.2.4. Algorytm SEC

N	czas - graf nieskierowany	std nieskierowany	czas - graf skierowany	std skierowany
100	5,52	0,876	9,898	0,252
200	11,643	1,557	19,459	0,951
300	17,371	0,799	30,827	3,036
400	23,729	0,969	38,598	2,649
500	30,742	1,976	47,283	0,952
600	35,92	0,742	57,109	1,311
700	44,063	3,807	65,774	2,748
800	49,024	1,912	77,069	7,373
900	57,017	4,06	83,749	1,667

1000	63,201	3,03	91,955	1,671
------	--------	------	--------	-------

8.3. Tabelki zależności czasu obliczeń t od liczby n wierzchołków w grafie i nasycenia s .

8.3.1. Algorytm SEC

8.3.1.1. Graf nieskierowany

	50%	60%	70%	80%	90%
100	6,605	5,761	6,514	6,274	6,597
200	11,879	12,977	14,209	14,865	14,94
300	18,389	19,054	20,362	22,131	23,913
400	23,77	28,187	27,343	29,863	31,237
500	30,647	33,34	35,554	37,058	39,917
600	36,354	42,134	43,573	44,403	48,593
700	46,525	47,854	50,116	56,723	55,493
800	50,199	53,301	57,341	61,309	66,224
900	56,58	61,046	63,94	68,89	72,456
1000	63,327	70,301	84,309	78,896	84,028

8.3.1.2. Graf skierowany

	50%	60%	70%	80%	90%
100	11,048	12,461	15,47	16,48	17,932
200	21,326	25,312	27,598	30,9	34,958
300	29,701	35,095	39,511	52,505	59,519
400	42,206	52,106	58,761	62,521	78,141
500	62,465	59,922	70,44	85,663	88,783
600	59,14	67,555	78,552	91,124	111,281
700	68,239	78,44	90,557	102,76	117,509
800	76,06	89,071	103,4	116,9	134,675
900	84,254	2,448	119,44	132,08	145,182
1000	94,227	13,481	126,57	146,16	163,799

8.3.2. Algorytm SHC

8.3.2.1. Graf nieskierowany

	50%	60%	70%	80%	90%
10	0,019	0,019	0,007	0,015	0,005
12	0,083	0,022	0,042	0,009	0,012
14	0,076	0,041	0,013	0,012	0,018
16	1,213	0,05	0,019	0,017	0,012
18	0,096	0,056	0,053	0,014	0,045
20	0,06	0,044	0,041	0,019	0,019
22	0,153	0,023	0,037	0,027	0,027
24	10,672	0,096	0,035	0,032	0,021
26	0,156	0,063	0,034	0,047	0,04
28	284,695	0,14	0,07	0,034	0,049

8.3.2.2. Graf skierowany

	50%	60%	70%	80%	90%
10	0,037	0,034	0,048	0,027	0,02
12	0,119	0,384	0,046	0,03	0,032
14	0,136	0,084	0,07	0,048	0,037
16	0,22	0,099	0,06	0,065	0,046
18	0,288	0,089	0,073	0,068	0,076
20	0,096	0,149	0,112	0,066	0,061
22	12,757	0,075	0,105	0,086	0,133
24	519,849	0,192	0,116	0,169	0,115
26	0,531	0,196	0,101	0,102	0,092
28	0,135	0,224	0,089	0,112	0,139

Algorytmy zakodowano w języku C++.

Sprzęt, na którym zostały wykonane pomiary:

Procesor: AMD Ryzen 7 4800H

RAM: 16GB DDR4 3200MHz

Karta Graficzna: NVIDIA GeForce 1650 4GB

System operacyjny:

Windows 11